Javascript Modules

Odoo supports three different kinds of javascript files:

- **plain javascript files** (no module system),
- **native javascript module**.
- **Odoo modules** (using a custom module system),

As described in the **assets management page**, all javascript files are bundled together and served to the browser. Note that native javascript files are processed by the Odoo server and transformed into Odoo custom modules.

Let us briefly explain the purpose behind each kind of javascript file. Plain javascript files should be reserved only for external libraries and some small specific low level purposes. All new javascript files should be created in the native javascript module system. The custom module system is only useful for old, not yet converted files.

Plain Javascript files

Plain javascript files can contain arbitrary content. It is advised to use the *iife* **immediately invoked function execution** style when writing such a file:

```
(function () {
  // some code here
  let a = 1;
  console.log(a);
})();
```

The advantages of such files is that we avoid leaking local variables to the global scope.

Clearly, plain javascript files do not offer the benefits of a module system, so one needs to be careful about the order in the bundle (since the browser will execute them precisely in that order).

> **Note**
>
> In Odoo, all external libraries are loaded as plain javascript files.

benefits of a better developer experience with a better integration with the IDE.

Let us consider the following module, located in *web/static/src/file_a.js* :

```
import { someFunction } from "./file_b";

export function otherFunction(val) {
    return someFunction(val + 3);
}
```

There is a very important point to know: by default Odoo transpiles files under `/static/src` and `/static/tests` into **Odoo modules**. This file will then be transpiled into an Odoo module that looks like this:

```
odoo.define('@web/file_a', ['@web/file_b'], function (require) {
'use strict';
let __exports = {};

const { someFunction } = require("@web/file_b");

__exports.otherFunction = function otherFunction(val) {
   return someFunction(val + 3);
};

return __exports;
)};
```

So, as you can see, the transformation is basically adding `odoo.define` on top and updating the import/export statements. This is an opt-out system, it's possible to tell the transpiler to ignore the file.

```
/** @odoo-module ignore **/
(function () {
  const sum = (a, b) => a + b;
  console.log(sum(1, 2));
)();
```

odoo docs

so, it will automatically be converted to an Odoo module.

```
/** @odoo-module **/
export function sum(a, b) {
  return a + b;
}
```

Another important point is that the transpiled module has an official name: *@web/file_a*. This is the actual name of the module. Every relative imports will be converted as well. Every file located in an Odoo addon *some_addon/static/src/path/to/file.js* will be assigned a name prefixed by the addon name like this: *@some_addon/path/to/file*.

Relative imports work, but only if the modules are in the same Odoo addon. So, imagine that we have the following file structure:

```
addons/
    web/
        static/
            src/
                file_a.js
                file_b.js
    stock/
        static/
            src/
                file_c.js
```

The file *file_b* can import *file_a* like this:

```
import {something} from `./file_a`;
```

But *file_c* need to use the full name:

```
import {something} from `@web/file_a`;
```

Aliased modules

in the project won't be able to require the module. Aliases are here to map old names with new ones by creating a small proxy function. The module can then be called by its new *and* old name.

To add such alias, the comment tag on top of the file should look like this:

```
/** @odoo-module alias=web.someName**/
import { someFunction } from './file_b';

export default function otherFunction(val) {
    return someFunction(val + 3);
}
```

Then, the translated module will also create an alias with the requested name:

```
odoo.define(`web.someName`, ['@web/file_a'], function(require) {
    return require('@web/file_a')[Symbol.for("default")];
});
```

The default behaviour of aliases is to re-export the `default` value of the module they alias. This is because "classic" modules generally export a single value which would be used directly, roughly matching the semantics of default exports. However it is also possible to delegate more directly, and follow the exact behaviour of the aliased module:

```
/** @odoo-module alias=web.someName default=0**/
import { someFunction } from './file_b';

export function otherFunction(val) {
    return someFunction(val + 3);
}
```

In that case, this will define an alias with exactly the values exported by the original module:

```
odoo.define(`web.someName`, ["@web/file_a"], function(require) {
    return require('@web/file_a');
});
```

odoo docs

> for example, three names to call the same module, you would have to add a proxy manually. This is not good practice and should be avoided unless there is no other options.

Limitations

For performance reasons, Odoo does not use a full javascript parser to transform native modules. There are, therefore, a number of limitations including but not limited to:

- an `import` or `export` keyword cannot be preceded by a non-space character,
- a multiline comment or string cannot have a line starting by `import` or `export`

```
// supported
import X from "xxx";
export X;
  export default X;
    import X from "xxx";

/*
 * import X ...
 */

/*
 * export X
 */


// not supported

var a= 1;import X from "xxx";
/*
  import X ...
*/
```

- when you export an object, it can't contain a comment

```
// supported
export {
  a as b,
```

```
export {
  a
} from "./file_a"


// not supported
export {
  a as b, // this is a comment
  c,
  d,
}

export {
  a /* this is a comment */
} from "./file_a"
```

- Odoo needs a way to determine if a module is described by a path (like *./views/form_view*) or a name (like `web.FormView`). It has to use a heuristic to do just that: if there is a `/` in the name, it is considered a path. This means that Odoo does not really support module names with a `/` anymore.

As “classic” modules are not deprecated and there is currently no plan to remove them, you can and should keep using them if you encounter issues with, or are constrained by the limitations of, native modules. Both styles can coexist within the same Odoo addon.

Odoo Module System

Odoo has defined a small module system (located in the file *addons/web/static/src/js/boot.js*, which needs to be loaded first). The Odoo module system, inspired by AMD, works by defining the function `define` on the global odoo object. We then define each javascript module by calling that function. In the Odoo framework, a module is a piece of code that will be executed as soon as possible. It has a name and potentially some dependencies. When its dependencies are loaded, a module will then be loaded as well. The value of the module is then the return value of the function defining the module.

As an example, it may look like this:

```
    var A = ...;

    return A;
});

// in file b.js
odoo.define('module.B', ['module.A'], function (require) {
    "use strict";

    var A = require('module.A');

    var B = ...; // something that involves A

    return B;
});
```

If some dependencies are missing/non ready, then the module will simply not be loaded. There will be a warning in the console after a few seconds.

Note that circular dependencies are not supported. It makes sense, but it means that one needs to be careful.

Defining a module

The `odoo.define` method is given three arguments:

- `moduleName` : the name of the javascript module. It should be a unique string. The convention is to have the name of the odoo addon followed by a specific description. For example, `web.Widget` describes a module defined in the `web` addon, which exports a `Widget` class (because the first letter is capitalized)

  If the name is not unique, an exception will be thrown and displayed in the console.

- `dependencies` : It should be a list of strings, each corresponding to a javascript module. This describes the dependencies that are required to be loaded before the module is executed.

- finally, the last argument is a function which defines the module. Its return value is the value of the module, which may be passed to other modules requiring it.

```
    var ajax = require('web.ajax');

    // some code here
    return something;
});
```

If an error happens, it will be logged (in debug mode) in the console:

- `Missing dependencies` : These modules do not appear in the page. It is possible that the JavaScript file is not in the page or that the module name is wrong
- `Failed modules` : A javascript error is detected
- `Rejected modules` : The module returns a rejected Promise. It (and its dependent modules) is not loaded.
- `Rejected linked modules` : Modules who depend on a rejected module
- `Non loaded modules` : Modules who depend on a missing or a failed module

Get Help

Contact Support | Ask the Odoo Community